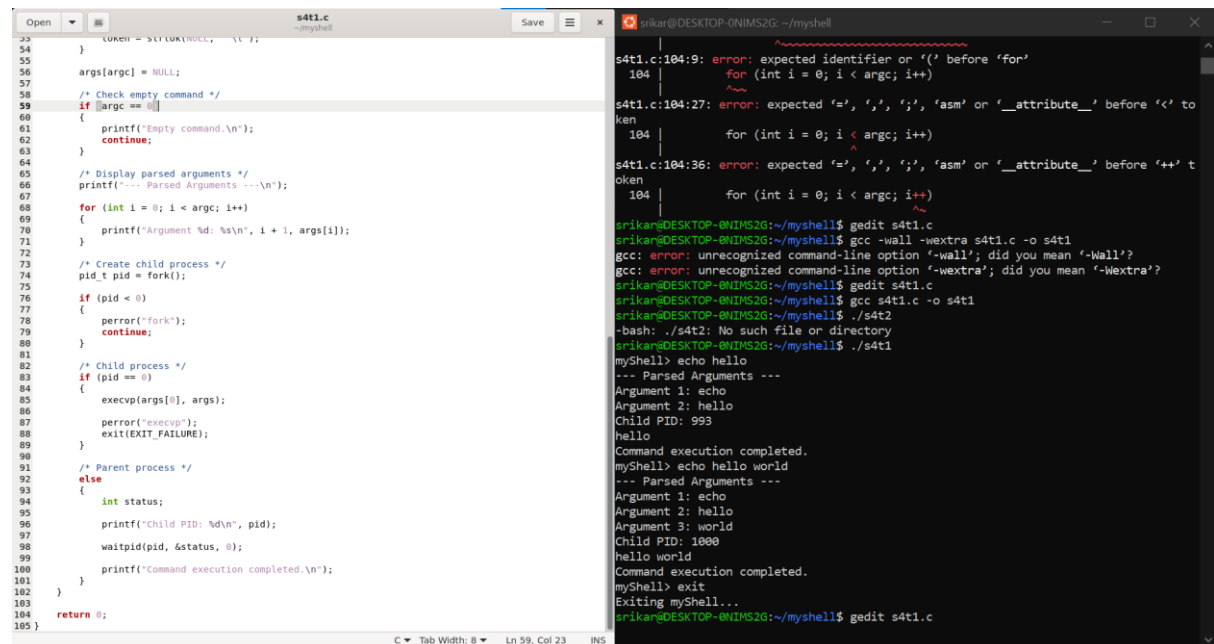


Skill -4:

Task 1:-



The screenshot shows a C program named `s4t1.c` in a text editor and its execution in a terminal. The program is a simple shell that parses arguments and executes them. The terminal output shows the program being compiled with `gcc` and then executed. It demonstrates parsing arguments like `echo` and `hello`, and executing them. The program also shows error handling for unrecognized command-line options like `-wall` and `-wextra`.

```
s4t1.c
/*
 * s4t1.c
 *
 * This program is a simple shell that parses arguments and executes them.
 * It uses the execvp function to execute the command specified in the first
 * argument.
 */
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <errno.h>

int main(int argc, char *argv[])
{
    /* Check empty command */
    if (argc == 0)
    {
        printf("Empty command.\n");
        continue;
    }

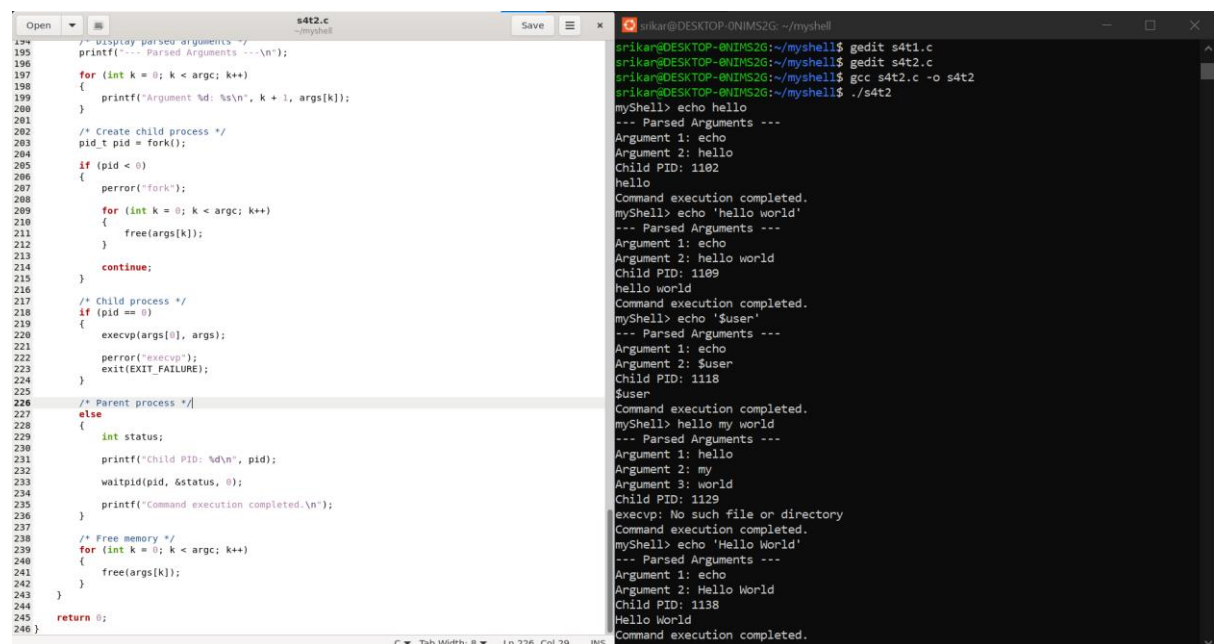
    /* Display parsed arguments */
    printf("--- Parsed Arguments ---\n");
    for (int i = 0; i < argc; i++)
    {
        printf("Argument %d: %s\n", i + 1, argv[i]);
    }

    /* Create child process */
    pid_t pid = fork();
    if (pid < 0)
    {
        perror("fork");
        continue;
    }
    if (pid == 0)
    {
        /* Child process */
        if (argc == 0)
        {
            exit(EXIT_FAILURE);
        }
        execvp(argv[0], argv);
        perror("execvp");
        exit(EXIT_FAILURE);
    }
    else
    {
        /* Parent process */
        int status;
        printf("Child PID: %d\n", pid);
        waitpid(pid, &status, 0);
        printf("Command execution completed.\n");
    }
}

return 0;
```

```
s4t1.c:104:9: error: expected identifier or '(' before 'for'
104     for (int i = 0; i < argc; i++)
      ^
s4t1.c:104:27: error: expected '=', ',', ';', 'asm' or '__attribute__' before '<' token
104     for (int i = 0; i < argc; i++)
                          ^
s4t1.c:104:36: error: expected '=', ',', ';', 'asm' or '__attribute__' before '++' token
104     for (int i = 0; i < argc; i++)
                                    ^
srikar@DESKTOP-0NIMS2G:~/myshell$ gcc s4t1.c
gcc: error: unrecognized command-line option '-wall'; did you mean '-Wall'?
gcc: error: unrecognized command-line option '-wextra'; did you mean '-Wextra'?
srikar@DESKTOP-0NIMS2G:~/myshell$ gcc s4t1.c
srikar@DESKTOP-0NIMS2G:~/myshell$ ./s4t1
-bash: ./s4t2: No such file or directory
srikar@DESKTOP-0NIMS2G:~/myshell$ ./s4t1
myShell> echo hello
--- Parsed Arguments ---
Argument 1: echo
Argument 2: hello
Child PID: 993
hello
Command execution completed.
myShell> echo hello world
--- Parsed Arguments ---
Argument 1: echo
Argument 2: hello
Argument 3: world
Child PID: 1000
hello world
Command execution completed.
myShell> exit
Exiting myShell...
```

Task 2:-



The screenshot shows a C program named `s4t2.c` in a text editor and its execution in a terminal. The program is a simple shell that parses arguments and executes them. It uses the `execvp` function to execute the command specified in the first argument. The terminal output shows the program being compiled with `gcc` and then executed. It demonstrates parsing arguments like `echo` and `hello`, and executing them. The program also shows error handling for unrecognized command-line options like `-wall` and `-wextra`.

```
s4t2.c
/*
 * s4t2.c
 *
 * This program is a simple shell that parses arguments and executes them.
 * It uses the execvp function to execute the command specified in the first
 * argument.
 */
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <errno.h>

int main(int argc, char *argv[])
{
    /* Display parsed arguments */
    printf("--- Parsed Arguments ---\n");
    for (int k = 0; k < argc; k++)
    {
        printf("Argument %d: %s\n", k + 1, argv[k]);
    }

    /* Create child process */
    pid_t pid = fork();
    if (pid < 0)
    {
        perror("fork");
        continue;
    }
    if (pid == 0)
    {
        /* Child process */
        if (argc == 0)
        {
            exit(EXIT_FAILURE);
        }
        execvp(argv[0], argv);
        perror("execvp");
        exit(EXIT_FAILURE);
    }
    else
    {
        /* Parent process */
        int status;
        printf("Child PID: %d\n", pid);
        waitpid(pid, &status, 0);
        printf("Command execution completed.\n");
    }

    /* Free memory */
    for (int k = 0; k < argc; k++)
    {
        free(argv[k]);
    }
}

return 0;
```

```
srikar@DESKTOP-0NIMS2G:~/myshell$ gcc s4t1.c
srikar@DESKTOP-0NIMS2G:~/myshell$ gcc s4t2.c
srikar@DESKTOP-0NIMS2G:~/myshell$ gcc s4t2.c -o s4t2
srikar@DESKTOP-0NIMS2G:~/myshell$ ./s4t2
myShell> echo hello
--- Parsed Arguments ---
Argument 1: echo
Argument 2: hello
Child PID: 1102
hello
Command execution completed.
myShell> echo 'hello world'
--- Parsed Arguments ---
Argument 1: echo
Argument 2: hello world
Child PID: 1109
hello world
Command execution completed.
myShell> echo '$user'
--- Parsed Arguments ---
Argument 1: echo
Argument 2: $user
Child PID: 1118
$user
Command execution completed.
myShell> hello my world
--- Parsed Arguments ---
Argument 1: hello
Argument 2: my
Argument 3: world
Child PID: 1129
execvp: No such file or directory
Command execution completed.
myShell> echo 'Hello World'
--- Parsed Arguments ---
Argument 1: echo
Argument 2: Hello World
Child PID: 1138
Hello World
Command execution completed.
```

Task 3:-

```
Open s4t3.c ~myshell Save x
303 /*
304  * Create child process
305  */
306 pid_t pid = fork();
307 if (pid < 0)
308 {
309     perror("fork");
310     for (int k = 0; k < argc; k++)
311     {
312         free(args[k]);
313     }
314     continue;
315 }
316 /*
317  * Child process
318  */
319 if (pid == 0)
320 {
321     execvp(args[0], args);
322     perror("execvp");
323     exit(EXIT_FAILURE);
324 }
325 /*
326  * Parent process
327  */
328 else
329 {
330     int status;
331     printf("Child Process\n");
332     printf("PID = %d\n", pid);
333     waitpid(pid, &status, 0);
334     printf("Child process completed.\n");
335 }
336 /* Free memory */
337 for (int k = 0; k < argc; k++)
338 {
339     free(args[k]);
340 }
341 return 0;
342 }
```

```
Exiting myShell...
srikargDESKTOP-BNIMS2G:~/myshell$ gedit s4t2.c
srikargDESKTOP-BNIMS2G:~/myshell$ gedit s4t3.c
srikargDESKTOP-BNIMS2G:~/myshell$ gcc s4t3.c -o s4t3
srikargDESKTOP-BNIMS2G:~/myshell$ ./s4t3
myShell> echo hello
--- Parsed Arguments ---
Argument 1: echo
Argument 2: hello
Child Process
PID = 1230
hello
Child process completed.
myShell> echo "Hello World"
--- Parsed Arguments ---
Argument 1: echo
Argument 2: Hello World
Child Process
PID = 1241
Hello World
Child process completed.
myShell> Echo $user
--- Parsed Arguments ---
Argument 1: Echo
Argument 2: $user
Child Process
PID = 1248
execvp: No such file or directory
Child process completed.
myShell> echo
--- Parsed Arguments ---
Argument 1: echo
Child Process
PID = 1253
Child process completed.
myShell> exit
Exiting myShell...
srikargDESKTOP-BNIMS2G:~/myshell$ gedit s4t3.c
```

Task 4:-